



On the Importance of Gating: Memorization vs. In-Context Learning in State Space Models

William L. Tong^{2,*}, Aryo Lotfi¹, Emmanuel Abbe¹, Kostas Vaggelakos¹, Vishnu Banna¹, Etai Littwin¹, Josh Susskind¹, Cengiz Pehlevan², Eran Malach¹

¹Apple, ²Harvard University

*Work done while at Apple

State Space Models (SSMs) have emerged as a compelling alternative to Transformers, enabling sequence modeling with constant memory and linear compute. Although SSMs exhibit reasonable performance and favorable computational characteristics, they continue to lag behind Transformers on tasks that require in-context learning and precise retrieval, slowing their adoption for large-scale language modeling. In this work, we demonstrate that both the success and failure of SSMs in these domains can be explained by studying the role of the gating mechanism, a prevalent component in modern recurrent networks. Specifically, we show through theory and experiments that this gating mechanism causes SSMs to first learn an in-weights “memorization” solution, while delaying, or even preventing, convergence to a correct in-context learning solution. Importantly, this happens even in cases where there are no fundamental limitations due to the architecture or its memory capacity. On the other hand, we find that gating is often beneficial for improving generalization to long sequence lengths. Our results illuminate the crucial role of the gating mechanism in shaping both the training dynamics and generalization of SSMs, and provide a basis for understanding and improving linear-time models.

Correspondence: William L. Tong: wtong@g.harvard.edu; Eran Malach: e_malach@apple.com

Date: September 16, 2026

1 Introduction

In recent years, linear-time architectures like State Space Models (SSMs, e.g. [Gu et al. \(2021\)](#)) and variants of Linear Attention ([Katharopoulos et al., 2020](#)) have gained popularity for language modeling. Their primary benefit compared to Transformers is their memory and computational efficiency: their computational complexity grows linearly with the sequence length and their memory is constant, unlike Transformers that have quadratic computational complexity and linear scaling of memory¹. Different works have demonstrated that small and medium scale linear-time architectures can reach loss that is better than Transformers ([Gu and Dao, 2023](#); [Yang et al., 2024](#)), and can also achieve better length generalization ([Malach et al., 2025](#)). However, despite years of research, linear-time architectures are still not widely used for large scale language modeling². Indeed, several studies have identified fundamental limitations of linear-time architectures in performing in-context learning ([Waleffe et al., 2024](#)), copying ([Jelassi et al., 2024](#)), retrieval ([Arora et al., 2023](#)) and handling long-contexts ([Wang et al., 2025](#)). These observations may suggest that such architectures do not scale as well as Transformers, and that the computational efficiency comes at a significant cost in performance degradation for critical capabilities.

One common hypothesis as to why linear-time architectures lag behind Transformers is that there is simply

¹While a naive implementation of self-attention requires quadratic memory, efficient implementations such as FlashAttention ([Dao et al., 2022](#)) allow memory cost to be close to linear.

²While various recent models have adopted linear-time layers as part of a hybrid architecture ([Qwen, 2026](#); [Blakeman et al., 2025](#)), these models still rely on self-attention layers for enhanced retrieval capabilities.

an *expressivity* gap: SSMs operate with fixed-size memory, and therefore any capability that requires large memory will be sacrificed. In this work, we explore an alternative explanation. We show that in many cases, the initialization and training dynamics of SSMs tend to favor solutions that overly rely on in-weights memorization, even when the memory capacity is sufficient for solving the task using a correct in-context learning (ICL) solution. In particular, we show that the *gating* mechanism — prevalent in many modern linear-time architectures — biases SSMs to find memorizing solutions. At the same time, we show that the same gating mechanism enables SSMs to generalize better on sequences longer than the training data. Our main contributions are:

- We theoretically analyze the training dynamics of a simplified gated linear-time model, showing that, depending on the choice of the initial gating parameter, this model converges first to a “memorization” solution, even when a better ICL solution can be reached. On the other hand, we show that gating enables length generalization in some cases.
- We validate our theoretical findings in synthetic retrieval and logical rules tasks, showing that gated SSMs may fail to find the correct ICL solution. We show that changing the gating parameter at initialization results in faster convergence in some settings.
- To further validate our results, we finetune Mamba models on a tool-calling task with multiple tools presented in-context. We show that by tuning the initial gating parameter (after pretraining but before SFT), we can increase or decrease the hallucination rate, showing the important role of the gating parameter in in-context learning. Interestingly, we find that we can even improve performance by using a weaker gating than learned during pretraining.

Related work. Prior work finds that SSMs can learn in-context on synthetic tasks (Park et al., 2024; Grazzi et al., 2024; Li et al., 2025a), while pretrained models remain weaker on retrieval-intensive evaluations (Waleffe et al., 2024). Copying and retrieval also deteriorate with distance or sequence length (Jelassi et al., 2024; Arora et al., 2023). Most closely related to our mechanism, recurrent gating has been associated both with stable length extrapolation and with recency bias, while near-no-decay “mimetic” initialization improves synthetic copying (Wang et al., 2024; Trockman et al., 2024). We complement these results by clarifying the optimization competition between memorization and retrieval, and the related inference-time tradeoff between distant signal retention and distractor accumulation. Appendix A offers a more detailed review.

2 Theory: Retrieval with memorization “shortcuts”

We begin by presenting a tractable setting to study the fundamental principles governing the relationship between gating, retrieval, and generalization. Formal statements and proofs are deferred to Appendix B. We aim to construct a retrieval task that differentiates between *in-context learning* and *memorization* (or “in-weights learning”). In our setting, the model may use memorization to partially solve the task, obtaining non-trivial but imperfect accuracy; perfect performance requires in-context learning. We analyze the training dynamics of a simplified Mamba model.

Our analysis shows that the model either converges quickly to the ICL solution (when initialized with weak gating), or enters a plateau where memorization dominates before eventually recovering in-context learning (when initialized with strong gating). We then extend our analysis to long contexts, showing that strong gating may improve generalization. We validate our theoretical results through experiments in both our synthetic setting and more complex, naturalistic tasks. Overall, our results suggest the dual role of gating: stronger gating may induce memorization over ICL, but on the other hand improves performance on long contexts in certain settings.

2.1 Task

Let $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_P \in \mathbb{R}^d$ be random vectors sampled iid as $\mathbf{x}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I}/d)$. These points remain fixed through the duration of the task. To each of these P points, we assign a binary label $y(\mathbf{x}_i) = \pm 1$. For each example, we sample an ordered context of length ℓ and a query index q such that one context position contains $\mathbf{x}_q$.

The context reveals (possibly perturbed) labels, while the final query token reveals only the vector $\mathbf{x}_q$. We embed the resulting input as

$$\begin{pmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_\ell & \mathbf{x}_q \\ \tilde{y}(\mathbf{x}_1) & \tilde{y}(\mathbf{x}_2) & \cdots & \tilde{y}(\mathbf{x}_\ell) & 0 \end{pmatrix},$$

where $\tilde{y}(\mathbf{x}_i) = \eta_i y(\mathbf{x}_i)$ and, independently for each context occurrence, $\Pr(\eta_i = -1) = \epsilon$ and $\Pr(\eta_i = 1) = 1 - \epsilon$. Thus, the set of P points and their underlying labels remain fixed, but the perturbations are resampled for every example. The target is the *perturbed* label attached to the context occurrence of $\mathbf{x}_q$; the model needs to output $\tilde{y}(\mathbf{x}_q)$. If ϵ is small, then the model may succeed through simple *memorization* by learning the mapping $y(\cdot)$. Otherwise, in order to attain high accuracy, the model must find a match for $\mathbf{x}_q$ among the ℓ context points to deduce the correct (potentially perturbed) label. The specific choice of input embedding, in which we stack keys and labels, follows prior theoretical work on ICL (Zhang et al., 2024; Lu et al., 2025b), and lends itself to tractable, interpretable analysis. The task overall is inspired by synthetic formulations of in-context classification (Chan et al., 2022; Reddy, 2023).

2.2 Model

We study a simple Mamba-like SSM based on linear attention with a global gating parameter λ . Let $\tilde{\mathbf{x}}_t = (\mathbf{x}_t, \tilde{y}(\mathbf{x}_t)) \in \mathbb{R}^{d+1}$ be an augmented context token that includes the label, and let $a = -e^\lambda$ be the continuous time decay factor. Construct the hidden state as $\mathbf{h} = \tilde{\mathbf{x}}_q + \sum_{t=1}^{\ell} e^{a(\ell-t+1)} (\tilde{\mathbf{x}}_q^\top \tilde{\mathbf{x}}_t) \tilde{\mathbf{x}}_t$ and the model output as $f(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_\ell, \tilde{\mathbf{x}}_q) = \mathbf{w}^\top \mathbf{h}$, for some learnable readout $\mathbf{w} \in \mathbb{R}^{d+1}$. We train the model through gradient descent with binary cross entropy (i.e. logistic) loss $\mathcal{L}(f(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_\ell, \tilde{\mathbf{x}}_q), \tilde{y}(\mathbf{x}_q)) = \log(1 + \exp(-\tilde{y}f))$.

Our model is equivalent to a single-head S6 layer from Mamba 2, with key, query, and value matrices all set to the identity. The trainable parameters are the gating parameter λ and readout $\mathbf{w}$. We initialize $\mathbf{w} = \mathbf{0}$, $\lambda = \lambda_0$, and study the outcomes for different choices of λ_0 .

2.3 Short context retrieval

We first consider retrieval in the setting where d is very large and $P \ll d$. Under these conditions, the input tokens are effectively orthonormal: $\mathbf{x}_i \cdot \mathbf{x}_j \approx 0$ if $i \neq j$ and $\mathbf{x}_i \cdot \mathbf{x}_i \approx 1$. For this analysis, we assume that the tokens are exactly orthonormal and that $\epsilon \in (0, 1/2)$. Without loss of generality, we focus on the learning dynamics for a single token $\mathbf{x}_i$ and fix its context position to i . When clear that we are referring to $\mathbf{x}_i$, we drop the arguments for $y, \tilde{y}$ and f , which should be read as for instance $y \equiv y(\mathbf{x}_i)$ and $f \equiv f(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_\ell, \tilde{\mathbf{x}}_q)$ where $\mathbf{x}_q = \mathbf{x}_i$.

We decompose the model’s readout as $\mathbf{w} = (\mathbf{w}_x, v)$, where $\mathbf{w}_x \in \mathbb{R}^d$ is the weight vector associated to the input $\mathbf{x}$ and $v \in \mathbb{R}$ is the weight associated with the label $\tilde{y}$. To retrieve the label, the model needs to assign mass to weight v . A weight configuration with $\mathbf{w}_x = \mathbf{0}$ and $v = 1$ corresponds to a correct ICL solution. In-weights memorization corresponds to assigning non-zero weights to the input vector, thus memorizing the underlying mapping between inputs and values. We define $u = y(\mathbf{x}_i)(\mathbf{w}_x \cdot \mathbf{x}_i)$, which measures the model’s prediction according to the “memorized” label.

We denote by α the decay factor between the query token and the corresponding context input, which appears at position i , namely $\alpha = \exp(-(\ell - i + 1)e^\lambda)$. Note that strong gating (i.e. high λ) corresponds to low α , increasing the rate at which earlier tokens are decayed in the context. Weak gating (low λ) corresponds to high α , preserving more of the context.

We track the signed margin for point $\mathbf{x}_i$, which is given by $z = \tilde{y}f$. The signed margin tells us the degree to which a model classifies $\mathbf{x}_i$; a large, positive signed margin suggests the model classifies the point very confidently and correctly. We can show that $z = (1 + \alpha)\eta u + \alpha v$, so the signed margin naturally decomposes into two terms (see full details in Appendix B). We call $\delta_m = (1 + \alpha)u$ the *memorization margin* and $\delta_r = \alpha v$ the *retrieval margin*, so that $z = \eta\delta_m + \delta_r$. These margins quantify the degree to which the model is either memorizing or retrieving the label from context.

Proposition 1 (Behavior at initialization). *At initialization, the ratio of the initial growth rate of the retrieval margin to the initial growth rate of the memorization margin is $\frac{\dot{\delta}_r(0)}{\dot{\delta}_m(0)} = \frac{P\alpha^2}{(1-2\epsilon)(1+\alpha)^2}$.*

Proposition 1 analyzes the behavior of the model at initialization. For large α (weak gating), retrieval behavior dominates, and the model is able to quickly learn to retrieve the in-context label, solving the task. However, for small α (strong gating), memorization dominates, and the model is insensitive to the true label. This demonstrates that strong gating dampens in-context information, favoring memorization behavior. Proposition 1 suggests an approximate critical scale $\alpha^* \approx \sqrt{\frac{1-2\epsilon}{P}}$. If $\alpha > \alpha^*$ at initialization, then the model favors retrieval; if $\alpha < \alpha^*$, the model favors memorization. We compare this prediction with trained SSMs in the experiments shown in Figure 1a.

Next, we move beyond initialization to analyze training dynamics. Even if the model begins stuck memorizing, we find that it eventually escapes the memorization solution, with escape time depending on the initial value of α . We characterize the transition from memorization to retrieval by measuring the time at which $\delta_r > \delta_m$, i.e. when the signed margin is dominated by the retrieval margin. Our next proposition specifies the time at which this happens:

Proposition 4 (Informal: Escape time from memorization). *Let T be the first time at which $\delta_r(T) = \delta_m(T)$. Then, under the leading-order reduced dynamics on the memorization plateau and for small enough α_0 , $T = \Theta\left(\frac{1}{\alpha_0 \log(1/\alpha_0)}\right)$.*

A formal version of the result and its proof are given in Proposition 4; the assumed reduced dynamics are stated explicitly in equation B.2 and equation B.3. This result establishes that a model is not fatally trapped in a memorization solution, but given sufficient time, it will also learn the full retrieval solution. However, the required time may be immense. Since $\alpha = \exp(-(\ell - i + 1)e^\lambda)$, T may be exponential in the distance from the query $\ell - i + 1$, or indeed *double* exponential in the scale of the initialization for λ . Hence, to learn a retrieval task for which the relevant tokens are far from the query, an initialization with a very weak gating parameter λ (and therefore α close to 1) is essential to perform well in a reasonable amount of time. We validate these observations in a trained SSM in Figure 1b.

Interim summary In the short-context retrieval, the gating parameter λ controls the competition between memorization and retrieval. When gating is strong (and α is small), the label-coordinate readout v receives only a weak retrieval signal, while the token-specific component $\mathbf{w}_x$ can quickly learn the fixed label assignment. This creates a memorization plateau that can be escaped in time of order $\Theta((\alpha \log(1/\alpha))^{-1})$. Strong gating can delay in-context retrieval for a very long time, especially for tokens far from the query, even though the retrieval solution is eventually reachable. The short-context analysis therefore favors weak decay at initialization: preserving history makes it easier for gradient flow to discover the in-context solution before memorization dominates.

The next section shows why this conclusion may not continue to hold for long contexts, where preserving too much context through weak gating causes noise to accumulate from too many tokens.

2.4 Long context generalization

We next consider the setting where the number of examples in the sequence ℓ is large. We focus on studying retrieval when the model is trained on finite ℓ , then must generalize to arbitrary ℓ . To isolate retrieval, we now assume that the $\mathbf{x}_i$ are random Gaussian and that the labels are drawn uniformly s.t. $y_i \sim \{\pm 1\}$ (this corresponds to setting $P \rightarrow \infty$ and $\epsilon = 1/2$). In this setting, a model trained with finite ℓ learns a solution where $\mathbf{w}_x = 0$ and $v = \mathcal{O}(1)$ (Proposition 5). In the remainder of this section, we analyze the error of this solution as sequence length grows arbitrarily, identifying an “effective context length” determined by gating outside of which retrieval fails. For this analysis, we assume that the gating parameter λ is fixed at initialization. In practice, we find that gating remains close to its initialization in this setting, and our intuitions are representative of experiment (see Section B.3 for a more detailed discussion).

Denote $\tau = e^{-\lambda}$, which characterizes the effective context length (as we will later show). Note that strong gating corresponds to a smaller τ . Let $\text{Err}(\tau)$ be the error of the model given some choice of τ , namely $\text{Err}(\tau) = \Pr[f \neq \tilde{y}(\mathbf{x}_q)]$. The following result analyzes the error of the model at the no-decay (large τ) limit:

Proposition 8 (Informal: no-decay limit for long-context retrieval). *For retrieval with $\ell \geq 2$ input vectors of dimension d it holds that $\lim_{\tau \rightarrow \infty} \text{Err}(\tau) \approx \Phi(-\sqrt{d/\ell - 1})$.*

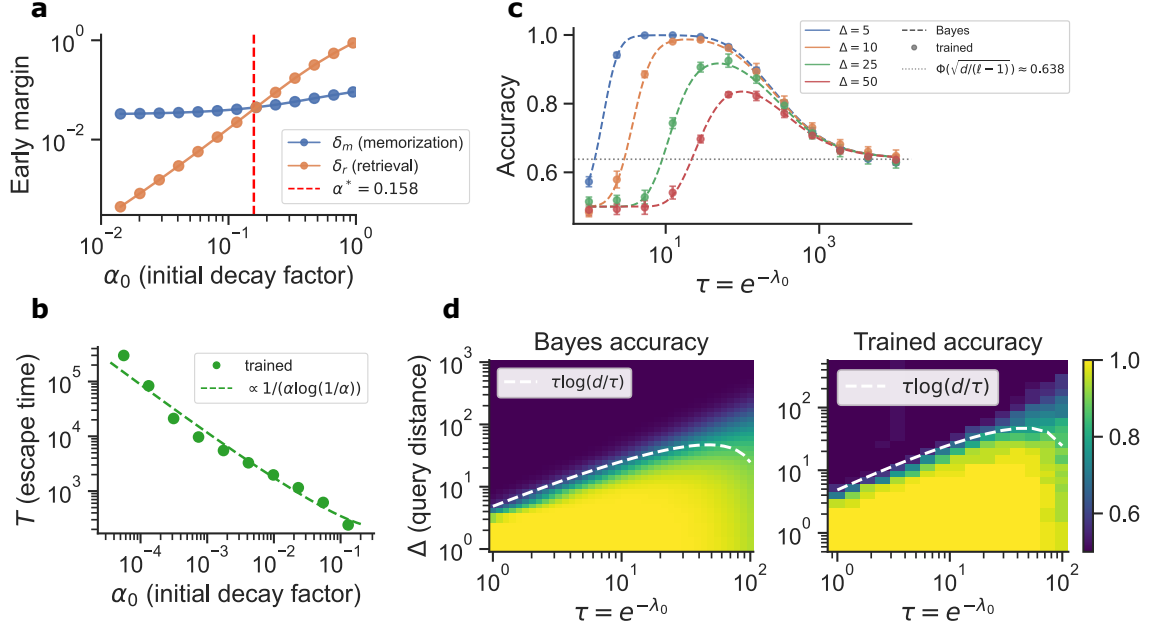


Figure 1 Validation of theoretical predictions in a simple trained SSM. We train the simple SSM described in Section 2.2 on our synthetic retrieval task, and verify that its behavior aligns with our theoretical predictions. **(a)** The critical α^* predicted from Proposition 1 determines whether the retrieval margin δ_r , or the memorization margin δ_m dominates at initialization. **(b)** The memorization plateau escape time measured from fully trained SSMs aligns closely with our predicted scaling from reduced dynamics in Proposition 4. **(c)** Bayes accuracy (dashed line) and experimentally measured accuracy (points) for varying τ . As τ increases, the accuracy asymptotes to the prediction from Proposition 8. **(d)** Heatmap comparing Bayes accuracy and accuracy of a trained SSM for varying τ and Δ . The contour predicted from Proposition 9 fits the measured accuracies well. Appendix D.1 enumerates the specific configurations for each experiment.

Proposition 8 (stated formally in Appendix B) shows that without decay, the relevant signal competes against all $L = \ell - 1$ distractors. We validate this prediction in Figure 1c. If $L/d \rightarrow \infty$ (in particular, if d is fixed and $L \rightarrow \infty$), then the no-decay signal-to-noise ratio converges to zero and the error approaches $\Phi(0) = 1/2$. This is the sense in which some decay is necessary for long-context retrieval: the model must reduce the effective number of retained distractors.

The next proposition gives more information about *which* positions specifically we have a chance to learn as context length increases and gating strengthens. We denote by $\text{Err}(\tau, \Delta)$ the error of the model on queries with Δ distance to the context input, i.e. $\text{Err}(\tau, \Delta) = \Pr[f \neq \hat{y}(\mathbf{x}_q) | \Delta = \ell - q + 1]$.

Proposition 9 (Decay induces locality). *Suppose $\ell/\tau \rightarrow \infty$, $1 \leq \tau = o(d)$. For any fixed $\delta \in (0, \frac{1}{2})$, let $\Delta_{\max}$ be the maximum distance from the query such that the model’s error is at most δ , i.e. the maximal Δ s.t. $\text{Err}(\tau, \Delta) \leq \delta$. Then $\Delta_{\max} = \Theta(\tau \log \frac{d}{\tau})$.*

Provided that $\tau \ll d$, Proposition 9 suggests that the distance of the furthest token away from which we can retrieve scales like $\tau \log(d/\tau)$, which we validate in Figure 1d. Thus τ controls the effective context size up to a logarithmic factor. Hence, generalization is only possible when we enable decay, but decaying induces a locality on our retrieval. Random retrieval is impossible as context length increases, but provided the relevant tokens remain close to the query, we may generalize to arbitrarily long contexts.

Together, the two regimes expose the basic tradeoff of gating. Weak gating makes it easier for gradient flow to favor retrieval over memorization in short contexts, but in long contexts it allows irrelevant tokens to accumulate enough variance to destroy the signal. Strong gating controls this variance and enables length generalization (within its effective context length τ), but may push the model into a long memorization plateau for short context retrieval.

3 Multi-Token Retrieval

In the remainder of this paper, we examine retrieval-based experiments that interrogate our theoretical intuitions in progressively more realistic settings. We test whether weaker gating promotes better in-context retrieval, and whether stronger gating promotes better length generalization. We begin by examining a simple multi-token retrieval task.

3.1 Setup

For a fixed N , each training sample presents a list of $n \sim \text{Unif}[1, N]$ key-value pairs in-context, followed by a set of query keys where each key/query consists of m tokens and the task is to predict the corresponding values of queried keys: $k_1 : v_1, k_2 : v_2, \dots, k_n : v_n | k_5 : ?, k_1 : ?, k_2 : ?$

More specifically, we choose a query ratio $q \in (0, 1)$ and train the model in two settings: *best-first*, where we query all the first $q \cdot n$ keys, and *best-last*, where we query all the last $q \cdot n$ keys. The best-first setting emphasizes in-context retrieval. From Section 2.3, we learned that increasing the distance between the target token and query amplifies the effect of gating by reducing the decay factor α . Hence, placing the relevant keys early in the context increases their distance to the query, amplifying the effect of gating. In contrast, the best-last setting emphasizes length generalization. From Section 2.4, we learned that strong gating improves length generalization but also imposes locality. We therefore predict that placing the relevant keys late in the context supports length generalization for strong gated models.

Like in our theoretical setting, we fix a static dictionary of key-value pairs shared across all examples; then for each in-context demonstration, we sample a pair from the dictionary and perturb each value token independently with probability ϵ . We set $N = 512$, $\epsilon = 0.1$, $q = 0.1$, and present each key/value using $m = 8$ tokens. Under these parameters, memorizing the static mapping yields sequence accuracy $(1 - \epsilon)^m \approx 0.43$; while achieving perfect accuracy requires in-context learning.

To smoothly interpolate between standard and no-decay initialization, we use an *offset* parameterization: we shift each head’s initial λ_0 by a constant $\Delta\lambda_0$, i.e., $\lambda_0 \leftarrow \lambda_0 + \Delta\lambda_0$. Setting $\Delta\lambda_0 = 0$ recovers the standard initialization, while $\Delta\lambda_0 \rightarrow -\infty$ approaches the no-decay regime. This allows us to study the effect of gating strength as a continuous parameter. See Appendix D.2 for details.

3.2 Results

Figure 2a and b present results for the best-first setting. When the gating parameter is small at initialization, the model learns the ICL solution. However, as the gating increases, the escape time from the memorization solution gets progressively longer. Further, Proposition 4 predicts an approximate scaling $T \propto 1/\alpha$ up to log factors for escape time T and decay factor α . Since $\alpha \propto e^{-\text{offset}}$, we may expect that $T \propto e^{\text{offset}}$. Indeed, panel b plausibly supports a linear relationship between T and e^{offset} .

In Figure 2c, we consider length generalization. Using the best-last setting, we evaluate how the gating initialization affects performance when the context is extended beyond the 512 key-value pairs observed during training. We observe that stronger gating at initialization leads to better length generalization, consistent with our theoretical results.

4 Logical Rules Retrieval

The tasks above are “pure” retrieval, where the model simply needs to match a pattern in the context. In practice, many settings involve more complex behavior, where the model needs to leverage information in the context to reason or act. To study more complex in-context retrieval, we consider agentic tool-use.

During a tool use interaction, the model is presented with a pool of potentially relevant tools. The model must then retrieve an appropriate tool that advances the user’s query. In-context retrieval is naturally essential as the model must determine from context which of many potentially novel tools to select. Length generalization is also important as the number of tools presented to the model may be variable. More tools implies a greater likelihood that a relevant one is present, but also increases the size of the context.

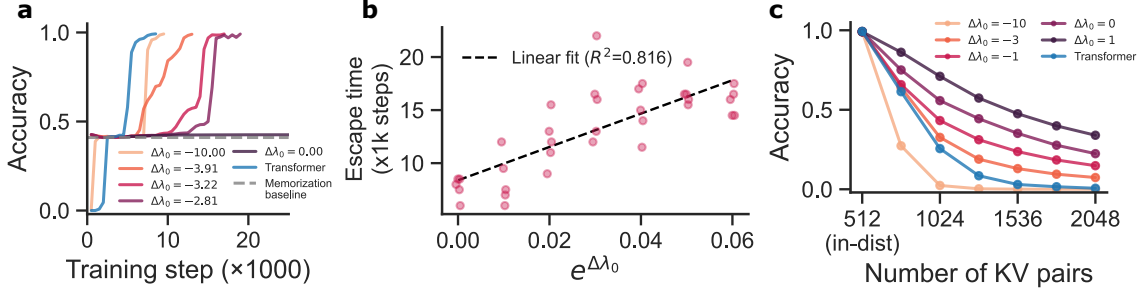


Figure 2 Performance on multi-token retrieval. (a) Accuracy across training steps for varying offset. Strong gating (large offset) never departs the memorization baseline within the training period. Smaller offset converge to perfect accuracy progressively faster. (b) Escape time across offsets. Proposition 4 predicts a linear scaling, which seems plausible in this experimental setting. (c) Accuracy across increasing context lengths. Models were trained on up to 512 KV pairs. Strong gating (large offset) generalizes better than weaker gating.

This section develops a synthetic task that measures retrieval over *Horn clauses*, logical statements with a convenient form for capturing knowledge and relationships. Our task is inspired by tool-calling in realistic settings while abstracting away many of the confounds.

4.1 Setup

A Horn clause is a logical statement that has the form $P_1 \wedge P_2 \wedge \dots \wedge P_n \rightarrow Q$, where the symbols P_i, Q are called *propositions*. Translated into English, this Horn clause reads “if P_1 through P_n are true, then Q ” is true. Horn clauses may be combined into longer deductions. For example, suppose I have clauses (1) $P_1 \rightarrow Q$ and (2) $P_2 \wedge Q \rightarrow R$. If P_1 and P_2 are true, then I may use (1) to derive Q , and (2) to derive R , so R must also be true.

Horn clauses are similar to function calls required in tool use. Like a function, a clause takes a set of inputs and produces an output. Suppose a user queries, “what is the weather right now in California?” The query embeds a set of propositions (“the time is 3:00pm,” “the location is California”). The model must then call the “get_weather” function, which may be represented as the Horn clause $(\text{time_is_3pm}), (\text{location_is_california}) \rightarrow (\text{weather_is_sunny})$. Horn clauses certainly do not capture all the nuances of function calls, but they offer a simple and controlled analogy that bridges our purely synthetic tasks above with the full tool use experiments we present in Section 5.

This task is constructed in the following way. We define M sets of propositions $\mathcal{S}_1, \dots, \mathcal{S}_M$. For each pair of adjacent sets $\mathcal{S}_i, \mathcal{S}_{i+1}$, we sample K Horn clauses with form $A \rightarrow B$ where $A \in \mathcal{S}_i$ and $B \in \mathcal{S}_{i+1}$. The prompt consists of a starting proposition $P \in \mathcal{S}_1$ and a goal proposition $Q \in \mathcal{S}_M$. The model must produce the correct sequence of Horn clauses to trace an intermediate sequence of propositions joining P and Q . Clauses are prompted in single turns: every turn, the model produces a single clause. The starting proposition is then updated to be the output of the chosen clause. Hence, to construct a chain from P to Q , the model outputs clauses through $M - 1$ rounds of interaction.

After being sampled, Horn clauses are fixed for the duration of the task. Each clause is assigned a random numeric name, which the model must output to select the corresponding clause. The mapping from names to clauses is presented in-context, and may vary between examples. Hence, a model must retrieve the appropriate clause in-context to advance the deduction, and cannot rely consistently on memorization of the name-to-clause association. During each turn, we select k clauses to present in context. If the current starting proposition is in set $\mathcal{S}_i$, then the input proposition of each context clause is also in $\mathcal{S}_i$ ³. Figure 3a illustrates the task.

We compare the performance of different Mamba models trained with varying gating strengths on this task. We also include a Transformer baseline. Specific details on training and task configuration are recorded in Appendix D.3.

³Since there are K total clauses transitioning from $\mathcal{S}_i$ to $\mathcal{S}_{i+1}$, we require that $k < K$.

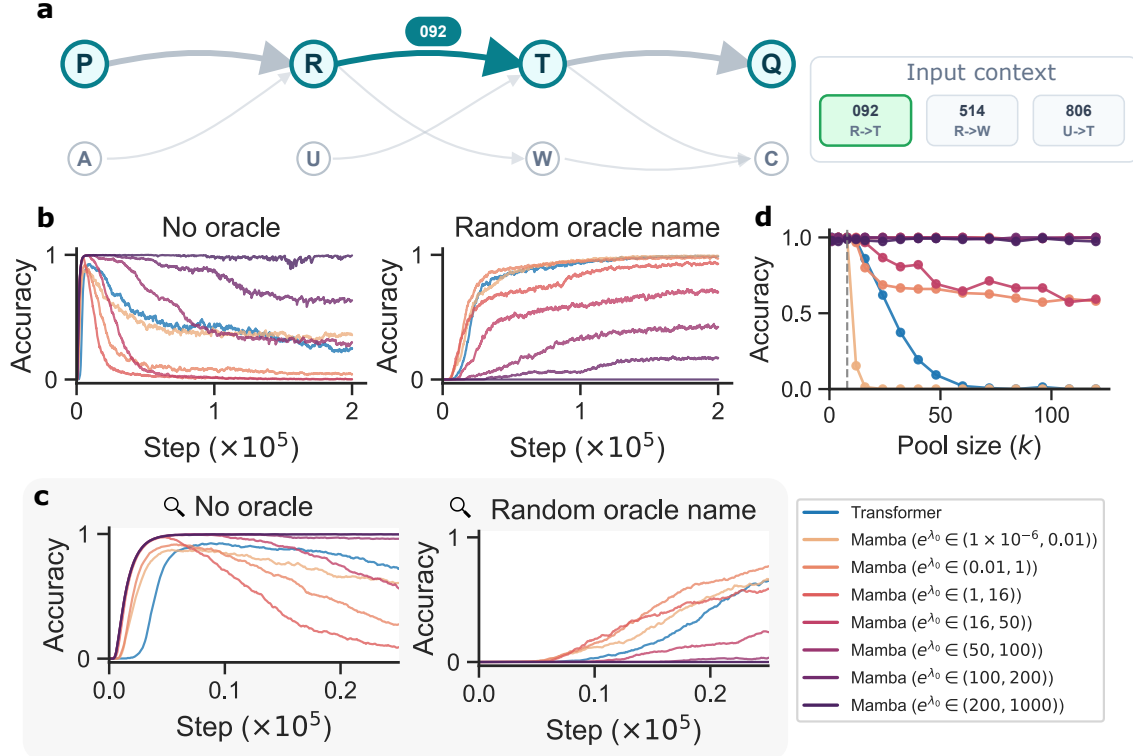


Figure 3 Performance on logical rules retrieval. (a) The logical rules retrieval task. Horn clauses transition between sets of propositions, with a starting proposition P and goal proposition Q . The model must select the correct clause by name from its context to advance the deduction. (b) In-distribution retrieval performance. “No oracle” tests memorization omitting the oracle clause in the context. “Random oracle name” tests retrieval by including an oracle clause in the context, but resampling its name. Weaker gating implies stronger retrieval performance and faster escape from the memorization plateau. (c) The same as in panel (b), but zoomed in to the start of training. (d) Length generalization performance. Stronger gating leads to stronger generalization.

4.2 Results

Figure 3 summarizes our results on the logic rules retrieval task. Our results support the intuition that weaker gating promotes in-context retrieval, whereas stronger gating benefits length generalization.

In-context retrieval. We compare retrieval performance on in-distribution context lengths, measured by the accuracy of the first turn⁴. Clause ordering is random, but an *oracle* clause is injected at a random position in every context during training. Similar to our prior setups, the mapping from names to clauses is static. However, names among the first $q = 0.1$ proportion of clauses are resampled per example, requiring the model to retrieve these from context rather than use their memorized association. This is analogous to the “best-first” setup from Section 3.

Figure 3b compares accuracy in two settings, and Figure 3c zooms into early training. “No oracle” plots accuracy when the oracle is not injected into the context. Hence, a model would attain high accuracy only if it memorizes the name-to-clause mapping. “Random oracle name” plots the accuracy on examples in which the oracle *is* injected, but its name is resampled⁵. A model attains high accuracy in this setting only if it learns to retrieve names in-context.

In general, we see that models with stronger gating tend to memorize for longer, maintaining high accuracy in the absence of an oracle but struggling to retrieve when its name is resampled. However, consistent with our theoretical intuition, the memorization phase appears to be a plateau that gives way to some degree of retrieval, where stronger decay delays the onset of retrieval behavior. For very weak gating ($e^{\lambda_0} < 1$), the model never learns a full memorization solution and never saturates accuracy in the absence of an oracle, consistent with an initial bias for learning retrieval.

Long context generalization. We next consider generalization to longer contexts. Clauses are ordered according to the ranking model described in Appendix D.3.1, where the most relevant clause appears closest to the query, similar to the “best-last” setup in Section 3. All names are resampled for every example, forcing the model to learn retrieval. Models are trained on examples with variable pool size up to $k = 8$, then tested on larger pools up to the maximum $K = 128$ clauses per transition. Figure 3d plots our length generalization results. Consistent with our intuition, models with weaker gating generalize poorly, with decaying performance at longer lengths. Models with stronger gating continue generalizing perfectly to lengths well beyond the training distribution.

5 Tool-Calling SSMs with Tool-Retrieval

To validate the intuitions developed in our theoretical setting and synthetic tasks, we study a naturalistic tool-calling task based on the Berkeley Function Calling Leaderboard (BFCL) (Patil et al., 2025). BFCL is a leading evaluation benchmark that measures a model’s ability to call tools. Given a natural language user query and pool of suggested tools, the model must execute a correct tool call. This setup furnishes a real-world tool use setting to validate our intuitions.

5.1 Setup

BFCL examples consist of a natural language user query together with a tool or list of tools to call. Tools are also presented with the arguments they take and a brief description of their function. The model must then decide on a tool, and output a well-formed function call, which is validated by a benchmark-specific deterministic checker. All parts of the function call must be correct in order for the example to pass. BFCL includes many different splits. We focus on evaluating the `python simple` split, where the tool calls consist of Python-style function invocations.

We extend BFCL to include variable tool pool sizes. By default, `python simple` presents only a single suggested tool. To enable variable pool sizes, we first collect all tools across all examples into a shared pool.

⁴It is possible that multiple paths exist from the current proposition to the goal. To account for these cases, first-turn accuracy considers a clause to be correct if the goal is reachable from the output proposition.

⁵The oracle with resampled name is injected within the first q proportion of clauses, matching the training setup.

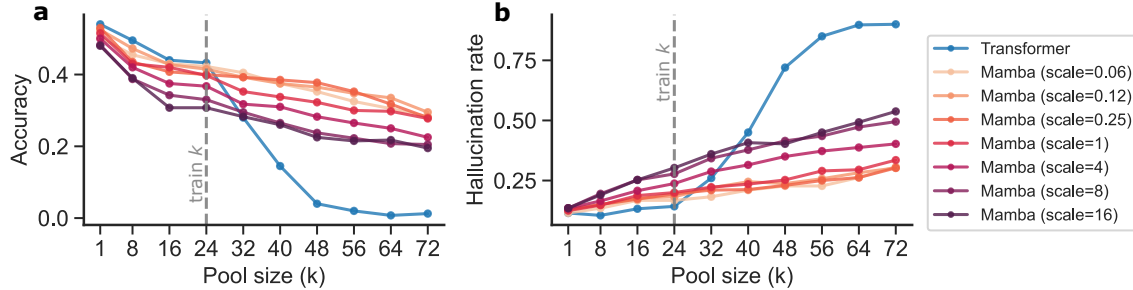


Figure 4 Evaluation on the BFCL benchmark. (a) Accuracy across tool pool sizes. Models were trained on tool pools with size up to $k = 24$. *Scale* is a factor s applied to Mamba’s decay rate before SFT (equivalently, $\lambda \leftarrow \lambda + \log s$ in our log-rate notation). Consistent with our intuition, models with weaker gating (lower scaling) perform better. Transformer performs well only on in-distribution lengths. (b) Hallucination rate counts examples where the model produces a syntactically valid tool call, but the tool does not exist in the input pool. Models with weaker gating also tend to hallucinate less, indicating stronger retrieval.

For each example, we then use BM25 (Robertson and Zaragoza, 2009) to rank the tools according to their relevance to the user query. Finally, we take the top k tools from the ranking and present them to the model. This procedure reflects standard retrieval-augmented tool-use pipelines, in which a retriever first narrows a large tool or API set before the LLM selects and invokes tools; sparse retrievers such as BM25 are common baselines in this setting (Patil et al., 2024; Qin et al., 2023). In cases where BM25 does not surface the correct tool for an example, we inject the correct tool at a random position in the context.

We evaluate a 2B parameter Transformer and 1.5B parameter Mamba models that were pretrained 300B tokens from the Nemotron-CC-HQ pretraining dataset (Su et al., 2025), then finetuned on the Nemotron Agentic tool-calling split (Basant et al., 2025). Aside from our custom tool pool system, we use the default BFCL evaluation harness to measure accuracy. See additional details on the experiment in Appendix D.4.

5.2 Results

Figure 4 summarizes our results on BFCL. All models were trained with variable pool size up to $k = 24$, then evaluated on larger pools. For the Mamba models, to control gating strength while also retaining skills learned during pretraining, we scale the gating parameter rather than re-initialize it. Larger scaling corresponds to stronger gating. Like in Section 3, the tool pool is ordered “best-first” so that the most relevant tool comes *first*, emphasizing the need for weak gating to preserve context.

We plot both the overall accuracy of each model in Figure 4a, as well as the hallucination rate in Figure 4b. Hallucination is measured by counting examples in which the model outputs a syntactically valid tool call, but the function does not appear among the provided pool. Consistent with our intuition, we see that models with weaker gating tend to score moderately better and hallucinate less, successfully retrieving the correct tool from their context more reliably. However, gating does not appear to influence length generalization in the range we probe. Rather, all Mamba models continue to length generalize better than the Transformer baseline, suggesting some benefit from even weak gating.

6 Discussion

In this work, we studied the role of the gating mechanism in linear-time architectures, focusing on the Mamba model. We showed that the initialization of the gating can affect training dynamics in SSMs, causing the model to prioritize memorization over in-context learning. On the other hand, we showed that gating improves length generalization in certain settings. While our results identify the gating mechanism as a crucial factor affecting in-context learning capabilities, our work does not immediately offer a solution for improving such capabilities, as our focus is primarily on scientific understanding of SSMs. We note that it is unclear whether simply changing the initialization, or even removing the gating mechanism, may be a good solution, as gating improves long-context performance in many regimes. However, it is possible that a

simple gating mechanism using multiplication by a scalar is not sufficient for complex memory management required in common retrieval problems. Instead, state-transition through higher-dimensional linear operators, or even non-linear transitions, might be required for optimal retrieval in long-context settings. We leave the investigation of these directions to future work.

Acknowledgments

WLT is supported by a Kempner Graduate Fellowship. CP is supported by an NSF CAREER Award (IIS-2239780), DARPA grants DIAL-FP-038 and AIQ-HR00112520041, the Simons Collaboration on the Physics of Learning and Neural Computation, and the William F. Milton Fund from Harvard University. This work has been made possible in part by a gift from the Chan Zuckerberg Initiative Foundation to establish the Kempner Institute for the Study of Natural and Artificial Intelligence. LLMs were used in implementing the experiments, checking proofs, and polishing the writing.

References

- Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Ré. Zoology: Measuring and improving recall in efficient language models. *arXiv preprint arXiv:2312.04927*, 2023.
- Aarti Basant, Abhijit Khairnar, Abhijit Paithankar, Abhinav Khattar, Adithya Renduchintala, Aditya Malte, Akhiad Bercovich, Akshay Hazare, Alejandra Rico, Aleksander Ficek, et al. Nvidia nemotron nano 2: An accurate and efficient hybrid mamba-transformer reasoning model. *arXiv preprint arXiv:2508.14444*, 2025.
- Assaf Ben-Kish, Itamar Zimerman, Shady Abu-Hussein, Nadav Cohen, Amir Globerson, Lior Wolf, and Raja Giryes. Decimamba: Exploring the length extrapolation potential of mamba. *arXiv preprint arXiv:2406.14528*, 2024.
- Sidney Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, et al. Gpt-neox-20b: An open-source autoregressive language model. In *Proceedings of BigScience Episode# 5-Workshop on Challenges & Perspectives in Creating Large Language Models*, pages 95–136, 2022.
- Aaron Blakeman, Aaron Grattafiori, Aarti Basant, Abhibha Gupta, Abhinav Khattar, Adi Renduchintala, Aditya Vavre, Akanksha Shukla, Akhiad Bercovich, Aleksander Ficek, et al. Nvidia nemotron 3: Efficient and open intelligence. *arXiv preprint arXiv:2512.20856*, 2025.
- Stephanie Chan, Adam Santoro, Andrew Lampinen, Jane Wang, Aaditya Singh, Pierre Richemond, James McClelland, and Felix Hill. Data distributional properties drive emergent in-context learning in transformers. *Advances in neural information processing systems*, 35:18878–18891, 2022.
- Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.
- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
- Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, et al. A survey on in-context learning. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pages 1107–1128, 2024.
- Riccardo Grazi, Julien Siems, Simon Schrod, Thomas Brox, and Frank Hutter. Is mamba capable of in-context learning? *arXiv preprint arXiv:2402.03170*, 2024.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2021.
- Ningyuan Huang, Miguel Sarabia, Abhinav Moudgil, Pau Rodriguez, Luca Zappella, and Federico Danieli. Understanding input selectivity in mamba: impact on approximation power, memorization, and associative recall capacity. *arXiv preprint arXiv:2506.11891*, 2025.

- Samy Jelassi, David Brandfonbrener, Sham M Kakade, and Eran Malach. Repeat after me: Transformers are better than state space models at copying. In *International Conference on Machine Learning*, pages 21502–21521. PMLR, 2024.
- Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon>, 6(3):4, 2024.
- Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.
- Yuval Koren, Assaf Ben-Kish, Raja Giryes, Lior Wolf, and Itamar Zimmerman. On the recall scaling laws in mamba: A theoretical and mechanistic study via hashing. *arXiv preprint arXiv:2609.07681*, 2026.
- Hongkang Li, Songtao Lu, Xiaodong Cui, Pin-Yu Chen, and Meng Wang. Can mamba learn in context with outliers? a theoretical generalization analysis. *arXiv preprint arXiv:2510.00399*, 2025a.
- Yingcong Li, Xupeng Wei, Haonan Zhao, and Taigao Ma. Can mamba in-context learn task mixtures? In *ICML 2024 Workshop on In-Context Learning*, 2024.
- Yingcong Li, Davoud Ataee Tarzanagh, Ankit Singh Rawat, Maryam Fazel, and Samet Oymak. Gating is weighting: Understanding gated linear attention through in-context learning. *arXiv preprint arXiv:2504.04308*, 2025b.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- Peng Lu, Jerry Huang, Qiuhaio Zeng, Xinyu Wang, Boxing Chen, Philippe Langlais, and Yufei Cui. Mamba modulation: On the length generalization of mamba. *arXiv preprint arXiv:2509.19633*, 2025a.
- Yue M Lu, Mary Letey, Jacob A Zavatone-Veth, Anindita Maiti, and Cengiz Pehlevan. Asymptotic theory of in-context learning by linear attention. *Proceedings of the National Academy of Sciences*, 122(28):e2502599122, 2025b.
- Eran Malach, Omid Saremi, Sinead Williamson, Arwen Bradley, Aryo Lotfi, Emmanuel Abbe, Josh Susskind, and Etai Littwin. To infinity and beyond: Tool-use unlocks length generalization in state space models. *arXiv preprint arXiv:2510.14826*, 2025.
- Destiny Okpekepe and Antonio Orvieto. When recalling in-context, transformers are not ssms. *arXiv preprint arXiv:2508.19029*, 2025.
- Jongho Park, Jaeseung Park, Zheyang Xiong, Nayoung Lee, Jaewoong Cho, Samet Oymak, Kangwook Lee, and Dimitris Papailiopoulos. Can mamba learn how to learn? a comparative study on in-context learning tasks. *arXiv preprint arXiv:2402.04248*, 2024.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024.
- Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *The twelfth international conference on learning representations*, 2023.
- Alibaba Qwen. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- Gautam Reddy. The mechanistic basis of data dependence and abrupt learning in an in-context classification task. *arXiv preprint arXiv:2312.03002*, 2023.
- Ruifeng Ren, Zhicong Li, and Yong Liu. Exploring the limitations of mamba in copy and cot reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 12550–12574, 2025.
- Stephen Robertson and Hugo Zaragoza. *The probabilistic relevance framework: BM25 and beyond*, volume 4. Now Publishers Inc, 2009.
- Ricardo Buitrago Ruiz and Albert Gu. Understanding and improving length generalization in recurrent models. *arXiv preprint arXiv:2507.02782*, 2025.

- Dan Su, Kezhi Kong, Ying Lin, Joseph Jennings, Brandon Norick, Markus Kliegl, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. Nemotron-cc: Transforming common crawl into a refined long-horizon pretraining dataset. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2459–2475, 2025.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- Asher Trockman, Hrayr Harutyunyan, J Zico Kolter, Sanjiv Kumar, and Srinadh Bhojanapalli. Mimetic initialization helps state space models learn to recall. *arXiv preprint arXiv:2410.11135*, 2024.
- Roger Waleffe, Wonmin Byeon, Duncan Riach, Brandon Norick, Vijay Korthikanti, Tri Dao, Albert Gu, Ali Hatamizadeh, Sudhakar Singh, Deepak Narayanan, et al. An empirical study of mamba-based language models. *arXiv preprint arXiv:2406.07887*, 2024.
- Peihao Wang, Ruisi Cai, Yuehao Wang, Jiajun Zhu, Pragya Srivastava, Zhangyang Wang, and Pan Li. Understanding and mitigating bottlenecks of state space models through the lens of recency and over-smoothing. *arXiv preprint arXiv:2501.00658*, 2024.
- Youjin Wang, Yangjingyi Chen, Jiahao Yan, Jiaxuan Lu, and Xiao Sun. Memmamba: Rethinking memory patterns in state space model. *arXiv preprint arXiv:2510.03279*, 2025.
- Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024.
- Wangjie You, Zecheng Tang, Juntao Li, Lili Yao, and Min Zhang. Revealing and mitigating the local pattern shortcuts of mamba. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 12156–12178, 2025.
- Zhihao Zhan, Jianan Zhao, Zhaocheng Zhu, and Jian Tang. Overcoming long-context limitations of state-space models via context-dependent sparse attention. *arXiv preprint arXiv:2507.00449*, 2025.
- Ruiqi Zhang, Spencer Frei, and Peter L Bartlett. Trained transformers learn linear models in-context. *Journal of Machine Learning Research*, 25(49):1–55, 2024.

A Related Work

In-Context Learning with SSMS In-context learning (ICL), the ability of language models to adapt their predictions based on examples in their context, has been extensively studied in Transformer-based models (Dong et al., 2024). More recently, different works have studied ICL in SSMS, showing that SSMS achieve competitive ICL behavior on many synthetic learning problems (Park et al., 2024; Grazzi et al., 2024), learn in-context with outliers (Li et al., 2025a) or learn from task mixtures (Li et al., 2024). While SSMS seem to perform well on ICL tasks when trained in toy settings, empirical results on large scale pretrained Mamba models indicates that they still lag behind Transformers on tasks that require retrieval and ICL abilities (Waleffe et al., 2024). Our work aims to bridge the gap between the positive ICL performance reported in toy settings and the limited ICL capabilities observed in real pretrained models. We show that when the data contains “memorization” shortcuts, where the model can memorize local patterns in-weights instead of leveraging in-context demonstrations, gated SSMS often converge to the “memorization” solution.

Copying and Retrieval with SSMS The capabilities of SSMS to perform in-context retrieval has been studied extensively in the literature, benchmarking these models using Multi-Query Associative Recall (Arora et al., 2023) and string copying (Jelassi et al., 2024). It has been shown, both empirically and theoretically, that SSMS are able to perform retrieval from relatively short sequences (Huang et al., 2025; Koren et al., 2026). However, retrieval and copying capabilities have been shown to deteriorate as sequence length increases (Jelassi et al., 2024; Ren et al., 2025; Zhan et al., 2025). While we cannot expect SSMS to perform accurate retrieval across arbitrarily long sequences due to their fixed-size memory, it is unclear whether they perform well even in cases where memory size is not the bottleneck. For example, Okpeke and Orvieto (2025) show that SSMS are much more sensitive to choice of training hyper-parameters compared to Transformers when trained on retrieval tasks. We analyze a retrieval task, showing that Mamba models often converge first to a “memorization” solution, even when the architecture can perfectly solve the retrieval problem.

Gating and Recency Bias in SSMS Most modern linear RNNs introduce a *gating* mechanism, which decays the history state, often depending on the token input. This gating has been shown to stabilize training and improve generalization to longer sequence lengths, and is present in Gated Linear Attention (Yang et al., 2023), Mamba (Gu and Dao, 2023; Dao and Gu, 2024), Gated DeltaNet (Yang et al., 2024), and more. While some works have shown the benefits of the gating mechanism for outlier rejection (Li et al., 2025a) and in-context learning (Li et al., 2025b), other works expose some failures due to how gating is performed. For example, some works find that SSMS exhibit “recency bias” (Wang et al., 2024) or local pattern shortcuts (You et al., 2025), causing them to focus more on recent tokens and thus degrading performance on long context retrieval and causing information loss Wang et al. (2025). Trockman et al. (2024) find that adopting “mimetic” initialization, which essentially disables the gating at initialization, improves learning of copying in a synthetic setting.

Length Generalization in SSMS The capability of SSMS to generalize to sequences longer than the ones they are trained on has been studied in different settings. Gu and Dao (2023) demonstrate that Mamba achieves dramatically better length generalization performance compared to Transformers on an induction heads task, while other works show that the length generalization of SSMS can be improved through simple modifications (Ben-Kish et al., 2024; Ruiz and Gu, 2025). More recently, Malach et al. (2025) show that SSMS equipped with tool-use achieve remarkable length generalization on various tasks. Lu et al. (2025a) relate length generalization capabilities of Mamba to state convergence as input length increases. Our work establishes a trade-off between performance on long sequences and in-context retrieval capabilities.

B Detailed Theory

In Section 2 we gave a brief and informal version of our main theoretical findings. Here we give more details on the theory, including formal statements of all the results, the derivation and the proofs.

B.1 Short context retrieval - extended version

Decompose $\mathbf{w} = (\mathbf{w}_x, v)$, where $\mathbf{w}_x \in \mathbb{R}^d$ and $v \in \mathbb{R}$. Define the following new variables:

$$\begin{aligned}\Delta &= \ell - i + 1, \\ \alpha_\Delta &= \exp(-\Delta e^\lambda), \\ u &= y(\mathbf{x}_i)(\mathbf{w}_x \cdot \mathbf{x}_i),\end{aligned}$$

and let $\tilde{y} = \eta y$, where $\Pr(\eta = 1) = 1 - \epsilon$ and $\Pr(\eta = -1) = \epsilon$. In this fixed-position calculation we abbreviate $\alpha = \alpha_\Delta$. Then because our tokens are orthonormal, our model further simplifies to

$$f = y(\mathbf{x}_i)[(1 + \alpha)u + \alpha\eta v].$$

A useful quantity to track is the signed margin for point $\mathbf{x}_i$, which is given by $z = \tilde{y}f$. The signed margin tells us the degree to which a model classifies $\mathbf{x}_i$; a large, positive signed margin suggests the model classifies the point very confidently and correctly. The signed margin simplifies to

$$z = (1 + \alpha)\eta u + \alpha v.$$

The signed margin naturally decomposes into two terms. We call $\delta_m = (1 + \alpha)u$ the *memorization margin* and $\delta_r = \alpha v$ the *retrieval margin*, so that $z = \eta\delta_m + \delta_r$.

The memorization margin represents the model's capacity to memorize the fixed label for $\mathbf{x}_i$. It derives from $\mathbf{w}_x$, the component of $\mathbf{w}$ that dots only $\mathbf{x}_i$, with no information about its context label. Its contribution to the signed margin is multiplied by η , so it is flipped when the context label mutates from the fixed assignment.

The retrieval margin represents the model's capacity to retrieve the in-context label. v is the component of $\mathbf{w}$ that dots only the label coordinate, with no information about the actual token. In contrast to the memorization contribution, δ_r is not multiplied by η , so its classification is consistent regardless of whether the label mutates.

Averaging over the label noise η , the gradient flow equations are as follows. The factor $1/P$ appears only in the equation for u , because u is specific to the point $\mathbf{x}_i$, while v and λ are shared parameters that receive symmetric contributions from all queried points.

$$\begin{aligned}\dot{u} &= \frac{1}{P}(1 + \alpha)[(1 - \epsilon)\sigma(-(\delta_m + \delta_r)) - \epsilon\sigma(\delta_m - \delta_r)], \\ \dot{v} &= \alpha[(1 - \epsilon)\sigma(-(\delta_m + \delta_r)) + \epsilon\sigma(\delta_m - \delta_r)], \\ \dot{\alpha} &= (\alpha \log \alpha)^2[(1 - \epsilon)(u + v)\sigma(-(\delta_m + \delta_r)) + \epsilon(v - u)\sigma(\delta_m - \delta_r)].\end{aligned}$$

Proposition 1 (Behavior at initialization). *At initialization, the ratio of the initial growth rate of the retrieval margin to the initial growth rate of the memorization margin is*

$$\frac{\dot{\delta}_r(0)}{\dot{\delta}_m(0)} = \frac{P\alpha^2}{(1 - 2\epsilon)(1 + \alpha)^2}. \quad (\text{B.1})$$

Proof. At initialization $u(0) = v(0) = 0$, so $\delta_m(0) = \delta_r(0) = 0$ and $\dot{\alpha}(0) = 0$. The gradient flow equations give

$$\dot{u}(0) = \frac{(1 + \alpha)(1 - 2\epsilon)}{2P} \quad \text{and} \quad \dot{v}(0) = \frac{\alpha}{2}.$$

Since $\delta_m = (1 + \alpha)u$ and $\delta_r = \alpha v$, the initial margin growth rates are

$$\dot{\delta}_m(0) = \frac{(1 + \alpha)^2(1 - 2\epsilon)}{2P} \quad \text{and} \quad \dot{\delta}_r(0) = \frac{\alpha^2}{2}.$$

Taking the ratio proves the claim. $\square$

Proposition 1 suggests the behavior of the model at initialization. For large α , retrieval behavior dominates at initialization, and the model is able to quickly learn to retrieve the in-context label, solving the task. However, for small α , memorization dominates at initialization, and the model is insensitive to the label. This corresponds to our intuition that strong gating dampens in-context information, favoring memorization behavior. For large P and small α , equation B.1 suggests an approximate critical scale $\alpha^* \approx \sqrt{\frac{1-2\epsilon}{P}}$ such that if at initialization, $\alpha > \alpha^*$, the model favors retrieval, whereas if $\alpha < \alpha^*$, the model favors memorization.

If a model favors memorization initially, is it permanently trapped in such behavior, or will it uncover the retrieval solution eventually? In the remainder of this section, we establish that memorization is indeed a *plateau* rather than a sink, but that escaping this plateau may take significant time.

At initialization, our gradient flow equations suggest that $\dot{u}(0) = \mathcal{O}(1/P)$, while $\dot{v}(0) = \mathcal{O}(\alpha)$ and $\dot{\alpha}(0) = 0$. If $\alpha \ll \frac{1}{P}$, then v remains effectively 0 while u grows to a finite quantity. Hence, we might reasonably model our learning dynamics by assuming that the SSM quickly attains a perfect memorization solution, then observe how v and α evolve.

Lemma 2 (Perfect memorization). *Fix $v = 0$ and α . Then the expected loss $\mathbb{E}_\eta[\mathcal{L}(f, \tilde{y})]$ has a unique minimizer at*

$$u^* = \frac{\kappa}{1 + \alpha},$$

where $\kappa = \log \frac{1-\epsilon}{\epsilon} = \sigma^{-1}(1 - \epsilon)$.

Proof. With $v = 0$, the signed margin is $z = \eta(1 + \alpha)u$. The expected loss as a function of u is

$$L(u) = (1 - \epsilon) \log(1 + \exp(-(1 + \alpha)u)) + \epsilon \log(1 + \exp((1 + \alpha)u)).$$

Differentiating,

$$L'(u) = (1 + \alpha) [\sigma((1 + \alpha)u) - (1 - \epsilon)].$$

Thus the only stationary point satisfies $\sigma((1 + \alpha)u) = 1 - \epsilon$, or equivalently $(1 + \alpha)u = \kappa$. Moreover,

$$L''(u) = (1 + \alpha)^2 \sigma((1 + \alpha)u) (1 - \sigma((1 + \alpha)u)) > 0,$$

so this stationary point is the unique minimizer. $\square$

Once the model attains the memorization manifold suggested by Proposition 2, namely $\delta_m = (1 + \alpha)u = \kappa$, the corresponding time derivatives for v and α are

$$\begin{aligned} \dot{v}|_{\delta_m=\kappa} &= \alpha \phi(\delta_r), \\ \dot{\alpha}|_{\delta_m=\kappa} &= (\alpha \log \alpha)^2 [v \phi(\delta_r) + u^*(\alpha) \psi(\delta_r)], \end{aligned}$$

where $u^*(\alpha) = \kappa/(1 + \alpha)$ and

$$\begin{aligned} \phi(\delta) &= (1 - \epsilon) \sigma(-(\kappa + \delta)) + \epsilon \sigma(\kappa - \delta), \\ \psi(\delta) &= (1 - \epsilon) \sigma(-(\kappa + \delta)) - \epsilon \sigma(\kappa - \delta). \end{aligned}$$

Lemma 3 (Reduced dynamics on the memorization plateau). *Fix $\epsilon \in (0, 1/2)$. There are constants $c, C > 0$, depending only on ϵ , such that for all $\delta \in [0, \kappa]$,*

$$c \leq \phi(\delta) \leq C \quad \text{and} \quad |\psi(\delta)| \leq C|\delta|.$$

Consequently, on the memorization manifold, whenever $0 \leq \delta_r \leq \kappa$,

$$\dot{\alpha} = (\alpha \log \alpha)^2 [v \phi(\delta_r) + \mathcal{O}(\alpha|v|)].$$

Proof. We have $\phi(0) = 2\epsilon(1 - \epsilon) > 0$ and $\psi(0) = 0$, using $\sigma(\kappa) = 1 - \epsilon$ and $\sigma(-\kappa) = \epsilon$. The bounds on ϕ follow because ϕ is continuous and strictly positive on the compact interval $[0, \kappa]$. The bound on ψ follows from the mean-value theorem and boundedness of ψ' on the same interval. Since $u^*(\alpha) \leq \kappa$ and $\delta_r = \alpha v$, the approximation for $\dot{\alpha}$ follows from the exact plateau equation above. $\square$

Thus, to leading order in α , we study the reduced dynamics

$$\dot{v} = \alpha \phi(\delta_r), \quad (\text{B.2})$$

$$\dot{\alpha} = (\alpha \log \alpha)^2 v \phi(\delta_r). \quad (\text{B.3})$$

The omitted term can affect constants and the late part of the trajectory, but the early portion of the plateau dominates the asymptotic escape time below. Note that $\dot{v} > 0$ and $\dot{\alpha} > 0$ after v becomes positive. Therefore, we expect that the model will transition eventually from memorization to retrieval. One way of characterizing this transition is by measuring the time at which $\delta_r > \delta_m$. When this is true, the signed margin shifts to retrieval rather than memorization. Our next proposition specifies the time scale at which this happens under the reduced dynamics.

Proposition 4 (Escape time from the memorization plateau). *Fix $\epsilon \in (0, 1/2)$ and let $\kappa = \log((1 - \epsilon)/\epsilon)$. Initialize $v(0) = 0$ and $\alpha(0) = \alpha_0 \in (0, 1)$. Let T be the first time at which $\delta_r(T) = \alpha(T)v(T)$ reaches the memorization margin κ . Then, under the dynamics given by equation B.2 and equation B.3, as $\alpha_0 \rightarrow 0$,*

$$T = \Theta\left(\frac{1}{\alpha_0 \log(1/\alpha_0)}\right). \quad (\text{B.4})$$

Proof. Let $b = \log(1/\alpha) > 0$ and $b_0 = \log(1/\alpha_0)$, so $\alpha_0 \rightarrow 0$ is equivalent to $b_0 \rightarrow \infty$. Before the hitting time, the retrieval margin satisfies $\delta_r = \alpha v \in [0, \kappa]$, and therefore $\dot{v} = \alpha \phi(\delta_r) > 0$. Hence we may use v as a clock. Dividing the two reduced equations gives

$$\frac{d\alpha}{dv} = \alpha(\log \alpha)^2 v.$$

Equivalently, $db/dv = -b^2 v$, and hence

$$\frac{1}{b(v)} = \frac{1}{b_0} + \frac{v^2}{2}, \quad \alpha(v) = \exp\left(-\frac{1}{b_0^{-1} + v^2/2}\right).$$

Let $v_T = v(T)$. We first check that the value of v at the transition is $\Theta(1)$. The retrieval margin $\delta_r(v) = \alpha(v)v$ is increasing because both $\alpha(v)$ and v are increasing. Since $\alpha(v) \leq 1$, reaching $\delta_r(v_T) = \kappa$ requires $v_T \geq \kappa$. Conversely, choose a constant V such that $V \exp(-2/V^2) > \kappa$. From

$$\frac{1}{b(V)} = \frac{1}{b_0} + \frac{V^2}{2} \geq \frac{V^2}{2},$$

we get $b(V) \leq 2/V^2$, and hence $\alpha(V) = e^{-b(V)} \geq \exp(-2/V^2)$. Thus the retrieval margin at $v = V$ satisfies $\delta_r(V) = \alpha(V)V > \kappa$, so the hitting time must occur by $v = V$ and $v_T \leq V$. Therefore v_T is bounded above and below by positive constants depending only on ϵ , uniformly as $\alpha_0 \rightarrow 0$.

On the whole interval before the transition, the retrieval margin satisfies $0 \leq \delta_r(v) = \alpha(v)v \leq \kappa$. Lemma 3 therefore lets us treat $\phi(\delta_r)$ as a constant up to multiplicative factors. Using $dt = dv/\dot{v}$ and $\dot{v} = \alpha(v)\phi(\delta_r)$,

$$T = \int_0^{v_T} \frac{dv}{\alpha(v)\phi(\alpha(v)v)} = \Theta\left(\int_0^{v_T} \exp\left(\frac{b_0}{1 + b_0 v^2/2}\right) dv\right).$$

Because v_T is bounded between positive constants, it remains only to estimate this integral.

For the lower bound, for all sufficiently large b_0 , the interval $[0, 1/b_0]$ lies inside $[0, v_T]$. On this interval, writing $x = b_0 v^2/2$, we have $x \leq 1/(2b_0)$. Therefore

$$b_0 - \frac{b_0}{1+x} = \frac{b_0 x}{1+x} \leq b_0 x \leq \frac{1}{2}.$$

In particular,

$$\frac{b_0}{1 + b_0 v^2/2} \geq b_0 - 1.$$

Hence the integral is at least e^{b_0-1}/b_0 .

For the upper bound, use $v_T \leq V$ and integrate over $[0, V]$. On $0 \leq v \leq \sqrt{2/b_0}$, using $(1+x)^{-1} \leq 1-x/2$ for $x \in [0, 1]$ gives

$$\frac{b_0}{1+b_0v^2/2} \leq b_0 \left(1 - \frac{1}{2} \cdot \frac{b_0v^2}{2}\right) = b_0 - \frac{b_0^2v^2}{4},$$

so this part contributes at most

$$e^{b_0} \int_0^\infty \exp(-b_0^2v^2/4) dv = \mathcal{O}(e^{b_0}/b_0).$$

On the remaining interval $\sqrt{2/b_0} \leq v \leq V$, the denominator $1+b_0v^2/2$ is at least 2, so the exponent is at most $b_0/2$. This tail contributes at most $Ve^{b_0/2} = o(e^{b_0}/b_0)$. Thus the integral is $\Theta(e^{b_0}/b_0)$. Since $e^{b_0} = 1/\alpha_0$, the claimed scaling follows. $\square$

In this way, we establish that a model is not necessarily trapped in a memorization solution forever, but given sufficient time, it will also learn the full retrieval solution. However, the required time may be immense. Since $\alpha = \exp(-me^\lambda)$, T may be exponential in the position of the token m , or indeed *double* exponential in the scale of the base initialization for λ . Hence, to learn a retrieval task for which the relevant tokens are far from the query, an initialization with very small λ (and therefore α close to 1) is essential to perform well in a reasonable amount of time.

B.2 Long context generalization - extended version

Let $\alpha_j = \exp(-je^\lambda)$ denote the decay weight at distance j from the query, and let $\Delta = \ell - t^* + 1$ be the distance from the query to the context occurrence of $\mathbf{x}_q$. In our long-context setting, we prioritize studying retrieval, and allow the label mapping $\tilde{y}(\cdot)$ to be random (equivalent to setting $\epsilon = 0.5$). We also drop our orthogonality assumption and allow $P \rightarrow \infty$.

Since labels are i.i.d. random, a model trained in this setting cannot memorize a stable mapping from inputs to labels, and must rely on the retrieval signal in the final coordinate of $\mathbf{h}$. We might therefore expect the readout $\mathbf{w} = (\mathbf{w}_x, v)$ to have the form $\mathbf{w}_x \approx \mathbf{0}$ and $v > 0$. We formalize this intuition in the following proposition.

Proposition 5 (Pure retrieval solution). *Consider the pure retrieval distribution used in the long-context setting: the context length ℓ and dimension d are finite, the inputs are iid $\mathbf{x}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I}/d)$, the context labels $\tilde{y}_i$ are iid uniform ± 1 variables independent of the inputs, and the query token equals one context input $\mathbf{x}_{i^*}$ with target $\tilde{y}_{i^*}$. Let $\tilde{\mathcal{L}}(\mathbf{w}_x, v, \lambda) = \mathbb{E}[\mathcal{L}(f, \tilde{y}_{t^*})]$ be the expected logistic loss, and run gradient flow on $\tilde{\mathcal{L}}$ initialized with $\mathbf{w}_x(0) = \mathbf{0}$ and $v(0) = 0$. Then for every training time $t > 0$,*

$$\mathbf{w}_x(t) = \mathbf{0} \quad \text{and} \quad v(t) > 0.$$

Proof. Write $\mathbf{h} = (\mathbf{h}_x, g)$, where g is the final coordinate. For any orthogonal matrix $\mathbf{U} \in \mathbb{R}^{d \times d}$, applying $\mathbf{U}$ to every input vector maps $\mathbf{h}_x$ to $\mathbf{U}\mathbf{h}_x$, while preserving all inner products and leaving g and the target $\tilde{y}_{t^*}$ unchanged. The Gaussian input distribution is rotationally invariant, so the expected loss is also invariant under $\mathbf{w}_x \mapsto \mathbf{U}\mathbf{w}_x$:

$$\tilde{\mathcal{L}}(\mathbf{w}_x, v, \lambda) = \tilde{\mathcal{L}}(\mathbf{U}\mathbf{w}_x, v, \lambda). \tag{B.5}$$

Let $\mathbf{a} = \nabla_{\mathbf{w}_x} \tilde{\mathcal{L}}(\mathbf{0}, v, \lambda)$. Differentiating equation B.5 at $\mathbf{w}_x = \mathbf{0}$ gives $\mathbf{a} = \mathbf{U}^\top \mathbf{a}$ for every orthogonal $\mathbf{U}$. Thus $\mathbf{a}$ is unchanged by every rotation or reflection. The only vector with this property is $\mathbf{0}$. Hence $\nabla_{\mathbf{w}_x} \tilde{\mathcal{L}}(\mathbf{0}, v, \lambda) = \mathbf{0}$, so gradient flow initialized at $\mathbf{w}_x(0) = \mathbf{0}$ preserves $\mathbf{w}_x(t) = \mathbf{0}$.

Along this trajectory, the model output is vg . Therefore

$$\dot{v} = \mathbb{E}[\tilde{y}_{t^*} g \sigma(-\tilde{y}_{t^*} vg)].$$

At $v = 0$,

$$\dot{v} = \frac{1}{2} \mathbb{E}[\tilde{y}_{t^*} g] = \frac{1}{2} \mathbb{E}[\alpha_\Delta \|\mathbf{x}_{t^*}\|^2] > 0,$$

because the distractor terms in g have mean zero. Thus $v(t)$ moves positive immediately and cannot cross from positive to negative. $\square$

In the remainder of this section, we analyze the error of a model in which $\mathbf{w}_x = 0$ and $v = 1$. As Proposition 5 suggests, this captures the classifier learned by gradient flow on the expected loss, up to an irrelevant positive rescaling of the output. We are particularly interested in the model's capacity for length generalization, so we study the case in which $\ell \rightarrow \infty$.

Let us denote the final coordinate of $\mathbf{h}$ as g (or equivalently, the output of the model when setting $\mathbf{w}_x = 0$ and $v = 1$). From the simplified model's definition, we see that

$$g = \alpha_\Delta \|\mathbf{x}_q\|^2 \tilde{y}(\mathbf{x}_q) + \sum_{t \neq t^*} \alpha_{\ell-t+1} (\mathbf{x}_q^\top \mathbf{x}_t) \tilde{y}(\mathbf{x}_t).$$

Conditioned on a query $\mathbf{x}_q$ and label $\tilde{y}(\mathbf{x}_q)$, we observe that g has the following distribution with respect to randomness in the input tokens:

$$g \mid \mathbf{x}_q, \tilde{y}(\mathbf{x}_q) = \mathcal{N} \left(\alpha_\Delta \|\mathbf{x}_q\|^2 \tilde{y}(\mathbf{x}_q), \frac{1}{d} \|\mathbf{x}_q\|^2 (S_2(\ell, \lambda) - \alpha_\Delta^2) \right),$$

where $S_2(\ell, \lambda) = \sum_{j=1}^{\ell} \alpha_j^2$.

Intuitively, we see that the margin on $\mathbf{x}_q$ is weighted by α_Δ . For weaker decay, the weight α_Δ is larger. At the same time, g is subject to variance that increases with S_2 . With weaker decay, S_2 also grows larger, drowning out the margin. Hence, success through this coordinate requires selecting a moderate decay that promotes a large margin while controlling its variance. This is in contrast to the picture in Section 2.3, where weaker decay was monotonically better.

To clarify this intuition, we rely on the following two lemmas. In Lemma 6, we establish the Bayes error rate for the scalar statistic g . In Lemma 7, we derive an asymptotic characterization of $S_2(\ell, \lambda)$, yielding an "effective context size" as a function of our gating parameter.

Lemma 6 (Label-coordinate Bayes error rate). *Given the scalar statistic g for a query $\mathbf{x}_q$, the Bayes-optimal predictor based on g is $\text{sign}(g)$. The conditional error is*

$$\Pr(\text{sign}(g) \neq \tilde{y}(\mathbf{x}_q) \mid \mathbf{x}_q) = \Phi \left(-\alpha_\Delta \|\mathbf{x}_q\| \sqrt{\frac{d}{S_2(\ell, \lambda) - \alpha_\Delta^2}} \right),$$

where Φ is the standard Gaussian CDF.

Proof. Conditional on $\mathbf{x}_q$ and $\tilde{y}(\mathbf{x}_q)$, the signal term in g is deterministic and equal to $\alpha_\Delta \|\mathbf{x}_q\|^2 \tilde{y}(\mathbf{x}_q)$. For every distractor $t \neq t^*$, the inner product $\mathbf{x}_q^\top \mathbf{x}_t$ is Gaussian with mean zero and variance $\|\mathbf{x}_q\|^2/d$, and multiplication by the independent label $\tilde{y}(\mathbf{x}_t)$ leaves this Gaussian distribution unchanged. The distractor terms are independent, so their sum is Gaussian with variance $\|\mathbf{x}_q\|^2 (S_2(\ell, \lambda) - \alpha_\Delta^2)/d$.

The likelihood of g under $\tilde{y}(\mathbf{x}_q) = 1$ is a Gaussian with mean $\alpha_\Delta \|\mathbf{x}_q\|^2$, while the likelihood under $\tilde{y}(\mathbf{x}_q) = -1$ has the same variance and the opposite mean. With equal priors, the Bayes decision boundary for g is therefore $g = 0$, so the Bayes predictor based on this statistic is $\text{sign}(g)$. The error probability is the probability that a Gaussian with mean $\alpha_\Delta \|\mathbf{x}_q\|^2$ and variance $\|\mathbf{x}_q\|^2 (S_2(\ell, \lambda) - \alpha_\Delta^2)/d$ is negative, yielding the displayed expression. $\square$

Lemma 6 formalizes our intuition from above, where larger α_Δ (and therefore weaker decay) decreases the label-coordinate Bayes error. However, weaker decay also increases S_2 , which in turn increases the error. Our arguments revolve around this key insight as we employ this lemma below.

Lemma 7 (Effective context size). *Let $\tau = e^{-\lambda}$. If $\tau \geq 1$ and $\ell/\tau \rightarrow \infty$, then*

$$S_2(\ell, \lambda) = \Theta(\tau).$$

Moreover, $S_2(\ell, \lambda) - \alpha_\Delta^2 = \Theta(\tau)$ uniformly over $\Delta \in \{1, \dots, \ell\}$.

Proof. Since $\alpha_j = \exp(-j/\tau)$,

$$S_2(\ell, \lambda) = \sum_{j=1}^{\ell} e^{-2j/\tau} = \frac{e^{-2/\tau}(1 - e^{-2\ell/\tau})}{1 - e^{-2/\tau}}.$$

For $\tau \geq 1$, set $x = 2/\tau \in (0, 2]$. The inequalities $x \leq e^x - 1 \leq \frac{e^2 - 1}{2}x$ imply

$$\frac{\tau}{e^2 - 1} \leq \frac{1}{e^{2/\tau} - 1} \leq \frac{\tau}{2}.$$

Since $\ell/\tau \rightarrow \infty$ implies $1 - e^{-2\ell/\tau} \rightarrow 1$, this proves $S_2(\ell, \lambda) = \Theta(\tau)$. For the second statement, the smallest value of $S_2(\ell, \lambda) - \alpha_\Delta^2$ is obtained by removing the largest term, so it is

$$\sum_{j=2}^{\ell} e^{-2j/\tau} = \frac{e^{-4/\tau}(1 - e^{-2(\ell-1)/\tau})}{1 - e^{-2/\tau}} = \Theta(\tau).$$

The matching upper bound follows from $S_2(\ell, \lambda) = \Theta(\tau)$. $\square$

Lemma 7 indicates that the size of $S_2(\ell, \lambda)$ grows similarly with τ . Since $S_2(\ell, \lambda)$ counts the contribution to variance from each input token in the context (weighted by the decay factor), τ can be thought of intuitively as the effective size of the context induced by a choice of gating λ . Higher λ implies more severe decay, which in turn reduces the size of τ ; this is consistent with the intuition that more severe decay causes the model to forget earlier tokens, reducing the effective size of the context. We make the relationship between τ and context explicit in Proposition 9 below.

Our next proposition makes the no-decay limit explicit.

Proposition 8 (No-decay limit for long-context retrieval). *Fix $\ell \geq 2$ and $\Delta \in \{1, \dots, \ell\}$. Let $\tau = e^{-\lambda}$ and let $p_{\tau, \Delta}(\mathbf{x}_q)$ denote the conditional label-coordinate Bayes error at distance Δ . Then, as $\tau \rightarrow \infty$,*

$$p_{\tau, \Delta}(\mathbf{x}_q) \rightarrow \Phi\left(-\|\mathbf{x}_q\| \sqrt{\frac{d}{\ell - 1}}\right).$$

Proof. By Lemma 6,

$$p_{\tau, \Delta}(\mathbf{x}_q) = \Phi\left(-\alpha_\Delta \|\mathbf{x}_q\| \sqrt{\frac{d}{S_2(\ell, \lambda) - \alpha_\Delta^2}}\right).$$

For fixed ℓ , $\alpha_j = e^{-j/\tau} \rightarrow 1$ for every $j \in \{1, \dots, \ell\}$. Therefore $\alpha_\Delta \rightarrow 1$ and

$$S_2(\ell, \lambda) - \alpha_\Delta^2 = \sum_{j=1}^{\ell} \alpha_j^2 - \alpha_\Delta^2 \rightarrow \ell - 1.$$

The claimed limit follows by continuity of Φ . $\square$

Proposition 8 shows that without decay, the relevant signal competes against all $L = \ell - 1$ distractors. If $L/d \rightarrow \infty$ (in particular, if d is fixed and $L \rightarrow \infty$), then the no-decay signal-to-noise ratio converges to zero and the Bayes error approaches $\Phi(0) = 1/2$. This is the sense in which some decay is necessary for long-context retrieval: the model must reduce the effective number of retained distractors. Lemma 7 says that in the regime $\ell/\tau \rightarrow \infty$, this effective number scales as $\Theta(\tau)$, so nontrivial label-coordinate retrieval requires τ not to be much larger than d . The next proposition gives more information about *which* positions specifically we have a chance to learn as context length increases and gating strengthens.

Proposition 9 (Decay induces locality). *Suppose $\ell/\tau \rightarrow \infty$, $1 \leq \tau = o(d)$, and $\|\mathbf{x}_q\| = \Theta(1)$. For any fixed $\delta \in (0, \frac{1}{2})$, let Δ_{max} be the maximum distance from the query such that the label-coordinate Bayes error for a token at Δ_{max} is at most δ . Then*

$$\Delta_{max} = \Theta\left(\tau \log \frac{d}{\tau}\right).$$

Proof. Let $c_\delta = -\Phi^{-1}(\delta) > 0$. By Lemma 6, the label-coordinate Bayes error at distance Δ is at most δ if and only if

$$\alpha_\Delta \|\mathbf{x}_q\| \sqrt{\frac{d}{S_2(\ell, \lambda) - \alpha_\Delta^2}} \geq c_\delta.$$

Lemma 7 gives $S_2(\ell, \lambda) - \alpha_\Delta^2 = \Theta(\tau)$ uniformly over Δ . Since $\|\mathbf{x}_q\| = \Theta(1)$ and $\alpha_\Delta = e^{-\Delta/\tau}$, there are constants $c_1, c_2 > 0$ depending only on δ and the implicit constants in $\|\mathbf{x}_q\| = \Theta(1)$ such that every retrievable distance satisfies

$$e^{-\Delta/\tau} \geq c_1 \sqrt{\frac{\tau}{d}},$$

and every distance satisfying $e^{-\Delta/\tau} \geq c_2 \sqrt{\tau/d}$ is retrievable. Taking logarithms gives matching upper and lower bounds of the form

$$\tau \left(\frac{1}{2} \log \frac{d}{\tau} - \log c_2 \right) \leq \Delta_{max} \leq \tau \left(\frac{1}{2} \log \frac{d}{\tau} - \log c_1 \right).$$

Because $\tau = o(d)$, the logarithm diverges, so the additive constants are lower order. Therefore $\Delta_{max} = \Theta(\tau \log(d/\tau))$. $\square$

Provided that $\tau \ll d$, Proposition 9 suggests that the distance of the furthest token away from which we can retrieve scales like $\tau \log(d/\tau)$. Thus τ controls the effective context size up to a logarithmic factor. Hence, generalization is only possible when we enable decay, but decaying induces a locality on our retrieval. Random retrieval is impossible as context length increases, but provided the relevant tokens remain close to the query, we may generalize to arbitrarily long contexts.

B.3 How representative is a static gate?

Our long-context analysis conditions on a fixed λ , whereas the gates in our full Mamba experiments remain trainable. The short-context dynamics give one reason initialization may nevertheless remain informative: the update to the gating factor contains $(\alpha \log \alpha)^2$ (see equation B.3), which becomes small near both weak gating ($\alpha \approx 1$) and strong gating ($\alpha \approx 0$).

We also directly measure gate drift in the logical-rules length-generalization experiment of Figure 3d. For each initialization, Table 1 reports the median effective length $\tau = -1/\log \alpha$ across heads before and after 200k training steps, together with the corresponding absolute drift in α . The effective lengths span five orders of magnitude at initialization. Their ordering is preserved after training, and the changes are modest relative to this sweep, although intermediate gates do move somewhat. Thus, initialization remains a useful proxy for the post-training gating regime.

Table 1 Gate drift during logical-rules length-generalization training. Medians are taken across Mamba heads in the experiment from Figure 3d.

Initial median τ (tokens)	Median τ after 200k steps	$ \Delta\alpha $
2.1×10^4 (weakest)	1.2×10^4	4×10^{-5}
297	315	1×10^{-3}
17	14	0.024
2.8	3.0	0.083
1.4	1.4	0.067
0.8	0.7	0.053
0.2 (strongest)	0.2	7×10^{-3}

C Weak Tool Retrieval

In our logical-rules retrieval (Section 4) and tool-calling (Section 5) tasks, we assume that the relevant Horn clause or tool definition always appears in the context. In practice, agentic systems use cheap, often noisy

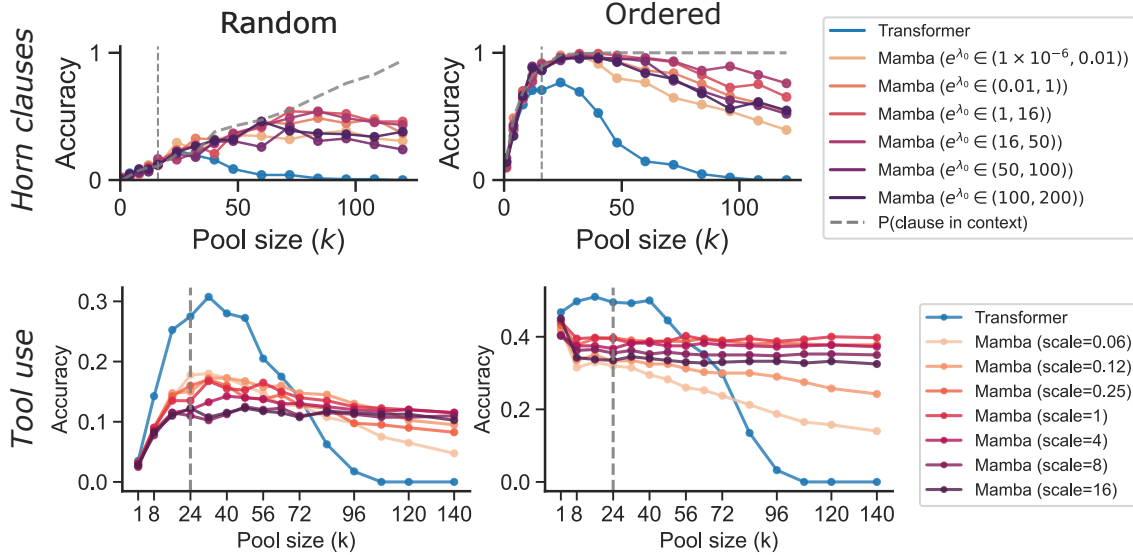


Figure 5 Accuracy with a weak tool retriever. We plot the accuracy of Mamba with a variety of gating strength together with a Transformer model on the logical rules retrieval task (*top*) and BFCL tool use benchmark (*bottom*), for both a virtually random tool retriever ($\beta = 0$ for Horn clauses, $\beta = 0.05$ for tool use) and noisily ordered tool retriever ($\beta = 1$ for Horn clauses, $\beta = 10$ for tool use). The scale of β differs between the two settings since their rankings have different scales. We see that Mamba models can generally improve or remain constant in performance with larger pool sizes, benefiting from greater recall that compensates for a weaker retriever. However, Transformers and Mamba models with low gating generalize less well, with performance falling as pool size increases.

tool retrievers to populate their context with relevant tools (Robertson and Zaragoza, 2009; Patil et al., 2024; Qin et al., 2023). As a result, the correct tool may not always be present.

In this appendix, we briefly highlight a setting in which we have a tool retriever whose fidelity is parameterized. A strong retriever surfaces the correct tool with high probability. A weak retriever surfaces the correct tool with low probability, but its weakness may be compensated by increasing the number of retrieved tools. In this way, we highlight another dimension in which balancing retrieval with length generalization is important: a model that generalizes to longer context lengths may accept a larger tool pool, compensating for a weaker retriever. Hence, an SSM with perfect retrieval but poor length generalization may not perform as well as an SSM that sacrifices some retrieval in favor of better length generalization.

C.1 Setup

To parameterize a retriever with controllable strength, we use the ranking procedure described in Section D.3.1, which uses a Plackett-Luce (PL) noise model with a temperature parameter β . When $\beta \rightarrow \infty$, the ranking is deterministic, such that the most relevant tool occurs first. When $\beta = 0$, ranking is purely random. Intermediate β interpolates between these two regimes, such that items with closer ranks experience a progressively greater probability of swapping as β decreases. We then take the top k items by rank to form our input context. Hence, higher β corresponds to stronger retrievers. We explore the performance of our models for varying β on both the logical rules retrieval and tool calling tasks below.

C.2 Results

See Figure 5 for results in our weak retrieval setting. In general, we find that a Mamba model with appropriately balanced gating strength generalizes well to larger pool sizes: low gating prevents the model from generalizing effectively to large pool sizes (and thus misses the greater recall afforded at large pool sizes), while strong gating prevents the model from learning retrieval as effectively. Moderate gating allows the model to continue generalizing effectively while also learning strong retrieval, benefiting most in this setting when the tool retriever is imperfect.

D Additional Experimental Details

Below, we record the particular model and task configurations in each experiment.

D.1 Theoretical validation experiments (Section 2)

We record additional details for the experiments used to validate our theoretical predictions, whose results are plotted in Figure 1. We train the simplified SSM model described in Section 2.2 using stochastic gradient descent with learning rate 0.1 and batch size 128; we use SGD here because it most directly matches our gradient-flow analysis. Additional panel-specific configurations are

- Figure 1a (initialization bias): We configured the task with $d = 512$, $P = 32$, $\ell = 8$, $\Delta = 4$, and $\epsilon = 0.1$. The margins were computed from the model parameters after 25 gradient steps.
- Figure 1b (escape time): We configured the task with $d = 512$, $P = 32$, $\ell = 4$, $\Delta = 1$, $\epsilon = 0.1$, and increased the learning rate to 0.3 to ensure escape time T fell within a tractable range.
- Figure 1c (Accuracy with increasing τ): We configured the task with $d = 128$, $\ell = 1024$, and trained for 10 thousand gradient descent steps before measuring accuracy.
- Figure 1d (Heatmap for varying τ and Δ). We configured the task with $d = 128$, $\ell = 512$, and trained for 10 thousand gradient descent steps before measuring accuracy.

D.2 Multi-token retrieval task (Section 3)

We give more details on the perturbed key-value experiments here. In all results, both keys and values are of length $m = 8$ tokens, drawn from a vocabulary of 64 tokens. We first generate a static dictionary D of key-value pairs, of size $|D| = 10000$. During training, we fix a maximal number of key-value pairs N , and for each example we sample $n \sim \text{Unif}(\{1, \dots, N\})$, sample $(k_1, v_1^*), \dots, (k_n, v_n^*) \sim D$ and then perturb each token in each static value v_i^* with probability ϵ , to generate an example with n in-context demonstrations $(k_1, v_1), \dots, (k_n, v_n)$. During evaluation, we only evaluate on the maximal choice of N for our in-distribution experiments. We use $\epsilon = 0.1$ and $q = 0.1$.

D.2.1 Model Architecture and Initialization

Mamba-2. We use Mamba-2 (Dao and Gu, 2024) with 24 layers, $d_{\text{model}} = 768$, expansion factor 2, 24 heads (head dimension 64), and SSM state dimension $d_{\text{state}} = 128$. Input and output embeddings are tied.

In Mamba-2’s default initialization, each head independently draws $\lambda \sim \log(\text{Uniform}(1, 16))$, giving one scalar per head with $\lambda \in [0, \log 16] \approx [0, 2.77]$. In the codebase, this parameter is referred to as $A_{\log}$. In the **offset** configuration, we shift all λ values by a constant δ after this default initialization:

$$\lambda \leftarrow \lambda + \delta.$$

Setting $\delta = 0$ recovers the standard initialization. Negative offsets uniformly reduce the decay, e.g., $\delta = -10$ shifts the range to approximately $[-10, -7.2]$, and effectively recovers the no-decay regime. This parameterization allows us to continuously vary the gating strength while retaining the per-head variation from the default initialization.

Transformer. We use GPT-NeoX (Black et al., 2022) with 12 layers, $d_{\text{model}} = 768$, 12 attention heads, intermediate size 3072, full RoPE (Su et al., 2024), Flash Attention 2, no dropout, and tied embeddings.

D.2.2 Optimization

All models are trained with AdamW (Loshchilov and Hutter, 2017) ($\beta = (0.9, 0.95)$), weight decay 0.1, peak lr 3×10^{-4} using linear warmup (5% of steps) followed by cosine decay to 10% of peak. Gradients are clipped to norm 1.0. Training uses bfloat16 mixed precision on 8 H100 GPUs with a global batch size of 128. We train for up to 100k steps with early stopping at 99% sequence accuracy.

D.2.3 Plot descriptions for Figure 2

All experiments in Figure 2 use five random seeds.

- **(a) In-distribution training curves.** For each offset value, we compute the *escape time* of each seed, defined as the first training step at which evaluation sequence accuracy exceeds 0.95. The plotted curve corresponds to the run with the median escape time among the five seeds. The default Mamba initialization ($e^{\text{offset}} = 1$) did not escape the memorization solution during our training (100k steps).
- **(b) Escape time vs. initialization.** For each offset, escape times of all five seeds are shown as individual points. The linear fit is shown with R^2 reported in the legend.
- **(c) Length generalization.** All models are trained on $N=512$ KV pairs and evaluated on $n \in \{512, 768, 1024, 1280, 1536, 1792, 2048\}$. For each offset and evaluation length, we report the mean final sequence accuracy across five seeds, with shaded bands indicating 95% confidence intervals.

D.3 Logic rules retrieval task (Section 4)

We provide additional details on the ranking procedure used to order clauses in context, and specific model/-task configurations.

D.3.1 Ranking clauses

During a tool use interaction, agentic systems rely on tool retriever algorithms to first collect a pool of tools relevant to the user query. This procedure is often done by using an efficient ranking algorithm like BM25 to order the set of all available tools, then select the top k to present to the model (Robertson and Zaragoza, 2009; Patil et al., 2024; Qin et al., 2023). Tool retrievers are often designed with efficiency in mind, with a tradeoff between speed and quality. Hence, the ranking procedure may involve a degree of noise.

To model this procedure in the context of our synthetic logical rules retrieval task, we consider the following process. For a transition from S_i to S_{i+1} , we sort the Horn clauses that implement this transition into four classes.

- **Class 1.** The clause is applicable given the current set of facts **and** the goal is reachable from the resulting state
- **Class 2.** The clause is applicable given the current set of facts but the goal is **not** reachable from the resulting state
- **Class 3.** The clause is **not** applicable given the current set of facts but the goal is reachable from the resulting state
- **Class 4.** The clause is **not** applicable given the current set of facts and the goal is **not** reachable from the resulting state

Given these classes, we order clauses probabilistically by class using a Plackett-Luce (PL) ranking model with a temperature parameter β . When $\beta = 0$, the order is random, and when $\beta \rightarrow \infty$, the order is deterministic in class, with class 1 clauses appearing first, followed by lower classes⁶. We then take the top k clauses in the order, and these become the pool of clauses presented in-context to the model. Hence, higher β corresponds to a stronger tool retriever that more accurately orders clauses and ensures a relevant clause is likely to appear in the context.

D.3.2 Model details

Across all experiments, we use a Mamba-2 architecture with 4 layers, 16 heads, and the same head and state dimension of 256. We use a Transformer with a Llama-like architecture featuring 4 layers and 8 heads, each with dimension 128. All models are trained using the Muon optimizer (Jordan et al., 2024) with peak learning rate 1×10^{-3} attained after a linear warm-up for 0.1 proportion of the total training time, followed by cosine decay. We use a batch size 128.

⁶The position of clauses *within* a class remains random

We use Muon because it trained both architectures substantially more reliably on this task; despite hyperparameter sweeps, AdamW did not train them consistently. Optimizer choice may affect the quantitative timing of the memorization-to-retrieval transition, although the qualitative dependence on gating is consistent across the SGD, AdamW, and Muon experiments in this paper.

D.3.3 Task details

The prompt has the following format:

```
name:clause|name:clause| . . . |start_prop,goal_prop<START>
```

name is a four character sequence of numerals that are randomly sampled for each clause. **clause** takes the form `r_abcd->r_efgh` where the letters correspond to alphanumeric symbols. In this way, we represent propositions as the character sequence `r_` followed by four alphanumeric symbols. **start_prop** is the starting/current proposition and **goal_prop** is the goal proposition. `<START>` is a special start token appended to the end of every prompt. `|` and `,` are special separator tokens. The model outputs a sequence of four numerals, corresponding to the name of the chosen clause.

Across all tasks, we restrict Horn clauses to have a single input proposition and a single output proposition. The task consists of $M = 3$ sets of propositions, necessitating two rounds of interaction. There are 128 propositions in each set, and $K = 128$ clauses transitioning between sets.

During training, the pool size is sampled uniformly at random between 1 and some maximum k (inclusive) per example. For the in-context retrieval experiments, the maximum pool size is fixed to be $k = 32$ during training. For the length generalization experiments, the maximum pool size is fixed to be $k = 8$ during training.

D.4 Tool-calling task (Section 5)

Training and Optimization Both pretraining and SFT use AdamW. For pretraining, we train on 300B tokens, with sequence length 4K and global batch size of 256. We use cosine decay, peak learning rate of 0.0003 and weight decay of 0.1. The Mamba and Transformer models use the same data and training pipeline.

Gate scaling Before SFT, we multiply Mamba’s decay rates e^λ by the scale s shown in Figure 4. In the log-rate parameterization used in Section 2.2, this intervention is exactly $\lambda \leftarrow \lambda + \log s$; it is therefore analogous to the additive offset used in the multi-token retrieval experiments.

Architecture For the SSM, we use a standard Mamba-2 architecture, with 32 layers and dimension $d = 2048$, totaling 1.5B parameters (including the embedding layer). For the Transformer, we use a Llama-based architecture, with 28 layers and dimension $d = 2048$, totaling 2B parameters (including the embedding layer).

D.5 Compute Resources

For all experiments in the paper, we run each individual training run on one node with 8xH100 Nvidia GPUs, each experiment takes below 10 hours, with a total of around 200 experiments. For pretraining the models, we train each model (Transformer and Mamba) on 128xH100, training for less than two days. We estimate that we used under 28,288 H100 GPU hours in total for this paper.